

Министерство цифрового развития, связи и массовых коммуникаций РФ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра вычислительных систем
Допустить к защите
Зав. кафедрой к.т.н., доцент
_____ Перышкова Е.Н.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Разработка 2D игры в жанре rogue-like

Пояснительная записка

Студент Исаков А.А.

Институт ИВТ Группа ИВ-123

Руководитель старший преподаватель Гонцова А.В.

Новосибирск - 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 Описание игры и средств разработки	9
1.1 Общее описание	9
1.2 Средства разработки	9
2 Структура игры.....	13
2.1 Общее описание	13
2.2 Подробное описание элементов игры.....	14
2.2.1 Класс Entity	14
2.2.2 Класс Planet	14
2.2.3 Класс Ship.....	14
2.2.4 Класс Unit и его наследники	15
2.2.5 Класс Entity_controller.....	16
2.2.6 Класс Enemy_controller	16
2.2.7 Класс Wave.....	17
2.2.8 Класс Wave_event.....	17
2.2.9 Класс Wave_event_spawn_enemy	17
2.2.10 Класс Wave_event_continuous_spawn_enemy	18
2.2.11 Структура Ship_spawn	18
2.2.12 Класс Wave_controller	18
2.2.13 Класс Game_controller.....	18
2.2.14 Основной файл Spaceroguelike.cpp.....	19
2.3 Вспомогательные алгоритмы.....	20
2.3.1 Алгоритмы чтения данных из файла, система прототипов и менеджеров	20
2.3.2 Создание текстур, анимации и звуков.	22
2.3.3 Создание пользовательского интерфейса.....	22
2.3.4 Вспомогательные функции	23
ЗАКЛЮЧЕНИЕ	24
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	25

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	26
ПРИЛОЖЕНИЕ А	27

ВВЕДЕНИЕ

Видеоигра – это программа, предназначенная для развлечения, обучения или тренировки, работающая на компьютере, игровых консолях, портативных устройствах или смартфонах. Это форма интерактивного взаимодействия человека с электронной системой, где пользователь управляет персонажем или объектом в виртуальном мире, следуя определенным правилам и целям.

Рынок видеоигр непрерывно растёт каждый год выходит множество крупных проектов как отечественных, так и из-за рубежа. В доказательство моим словам могу воспользоваться данными, которые предоставила аналитическая компания Newzoo в январе 2025 года о росте мирового рынка видеоигр до \$187,7 млрд в 2024 году, что на 2,1% превысило показатели 2023 года, однако не смогло превзойти рекордные значения 2021 года.

Для того, чтобы разобраться в причинах того, почему рынок игровой индустрии растет, следует разобраться в том, почему люди играют в игры.

Люди играют в компьютерные игры по следующим причинам:

Психологический отдых. Игра – это психологический отдых, который реализуется с помощью игрового процесса. Вовлечённость позволяет нам отдохнуть от работы и повседневной рутины.

Получение и тренировка навыков. Игры различных жанров способствуют прокачиванию ваших навыков, это не только развивает физические качества человека, такие как реакция, мелкая моторика и другие, но и повышает самооценку — веру в эффективность собственных действий и ожидание успеха от их реализации, одно из ключевых понятий социально-когнитивной теории научения Альберта Бандуры.

Снятие стресса. Игры помогают не только отдыхать, но и сбросить стресс. Активная вовлечённость в процесс позволяет отключиться от реальности.

Возможность получить новый уникальный опыт. Благодаря видеоиграм можно получить эмоции и впечатления, которые нельзя получить в реальной жизни, ввиду физических, финансовых или других возможностей.

Возможность поучаствовать в интересной истории. Благодаря тому что игры напрямую взаимодействуют с игроком, у них есть возможность погрузить в свой сюжет гораздо глубже, и изменить ход своего повествования в зависимости от его действий. Таким свойством обладают не только одиночные игры, нацеленные на сюжет, но и многопользовательские игры, где взаимодействие большого количества игроков приводит к тому, что внутри игр зарождаются интереснейшие истории, про масштабные сражения, коварные предательства или смешные ситуации, про которые можно рассказать друзьям.

Недалеко отходя от темы дружбы, игры помогают специализироваться, не только потому что у людей может быть общий к ним интерес, но и потому что большое количество игр построены на социальном взаимодействии, и побуждают к коммуникации и действию в команде.

Именно эти причины основные в вопросе почему люди играют, но для каждого эти причины могут быть своими, ведь существует огромное количество видеоигр, и каждый может найти в них что-то что именно ему по душе.

1 Описание игры и средств разработки

1.1 Общее описание

Жанр игры — vampire-like — поджанр rogue-like зародившийся совсем недавно — в 2022 году благодаря игре Vampire Survivors, отчего жанр и получил своё название, дословно: «Похожий на Vampire Survivors». Такие игры характеризуются тем, что игрок постоянно отбивается от толп врагов, в процессе становясь сильнее, но при поражении начиная всё заново. Целью игрока является либо прохождение какого-то фиксированного количества волн, либо продержаться как можно дольше, ставя рекорды, и соревнуясь с другими игроками.

Этот жанр я выбрал, потому что, при грамотной реализации и наполнении игры, игрок будет возвращаться в неё вновь и вновь. Чтобы пройти как можно дальше, или попробовать сыграть как-то по-другому.

Идея моей игры заключается в том, то игрок будет летать на космическом корабле, отбиваясь от врагов, защищая при этом не только себя, но и Землю от уничтожения. Сначала на игрока будут нападать заготовленные вручную волны, а после их прохождения у него будет возможность начать бесконечный режим со случайно сгенерированными атаками с нарастающей сложностью.

1.2 Средства разработки

Для игры был разработан собственный движок на основе C++ и библиотеки SFML версии 2.6.

C++ — это язык программирования, который был разработан в 1983 как расширение языка C. Этот язык отличается от C тем, что имеет больший набор возможностей, включая объектно-ориентированное программирование и шаблоны.

C++ используется для создания программного обеспечения разного рода: от игр до операционных систем. Этот язык также широко применяется в интенсивной обработке данных и научных расчетах. C++ предоставляет

разработчикам мощный и гибкий инструмент для создания программного обеспечения. Он позволяет писать эффективный и быстрый код, что делает его одним из наиболее популярных языков программирования в мире. C++ позволяет создавать приложения и программы любой сложности: от простых консольных утилит до сложных игровых движков. Также на этом языке можно программировать микроконтроллеры и системы в реальном времени.

На C++ можно написать практически все что угодно, от системных до мобильных приложений. Этот язык используется для создания операционных систем, программного обеспечения разного рода и игровых движков. Например, многие известные приложения были написаны на C++, включая операционные системы Windows и OS X, многие игры, такие как World of Warcraft и Counter-Strike. Благодаря C++ работают Unreal Engine 4, Microsoft Office и Adobe Photoshop.

Плюсы:

- Высокая производительность, потому что он не накладывает никакой избыточной нагрузки на программу, не использующую какие-либо возможности.
- Поддержка множества стилей программирования (процедурное программирование, абстракцию данных, объектно-ориентированное программирование и обобщенное программирование). Поэтому разработчик может сам выбрать, в каком стиле ему писать программу.
- Большое сообщество.

Минусы:

- Трудности с памятью. В C++ нужно самостоятельно управлять памятью, что может быть сложно для новичков. Например, нужно вручную выделять и освобождать память, чтобы избежать ошибок, таких как утечка памяти.
- Сложный синтаксис. C++ имеет много сложных конструкций, таких как указатели, ссылки и перегрузка операторов, которые могут запутать по началу. Понимание всех этих аспектов требует времени и опыта.

- Долгая компиляция. Процесс компиляции в C++ может занять много времени, особенно в больших проектах. Это может быть неудобно, если нужно часто вносить изменения в код и проверять результаты.

Библиотека SFML (Simple and Fast Multimedia Library) – это удобная и простая в использовании библиотека для разработки 2D-графики и мультимедийных приложений в C++. Она хорошо подходит для небольших проектов, как, например, игры или графические приложения, а также для обучения и быстрого прототипирования.

SFML предоставляет возможности не только работать с графикой, но и аудио, обработкой событий и нажатием кнопок.

SFML Graphics позволяет не только создавать и рисовать фигуры простые фигуры, но и использовать спрайты с текстурами, благодаря которым можно создавать продвинутую графику и анимации.

SFML Audio позволяет не реализовать как звуковые эффекты, так и фоновую музыку, библиотека так же позволяет создавать объёмное звучание и при необходимости запись аудио.

С помощью `sf::Event` можно отслеживать не только нажатие клавиш клавиатуры и мыши, но и изменение параметров окна приложения, что позволяет настроить систему, которая будет корректно отображать графику при любых параметрах окна.

Плюсы:

- Простота и быстрое освоение:

SFML отличается легкостью использования и не требует глубоких знаний в C++.

- Кроссплатформенность:

SFML работает под Windows, Linux, macOS и планируется выход под Android и iOS.

- Удобный API:

Простой и понятный API для работы с графикой, окнами, аудио, сетями и системой ввода.

- Легкость реализации графики:

Не требует написания больших объемов кода для создания графических элементов.

- Поддержка 2D графики:

Обеспечивает базовые возможности для создания 2D графических элементов, анимаций и эффектов.

- Большое сообщество и ресурсы:

Доступно множество онлайн-ресурсов, документации, примеров и сообществ, где можно найти помощь и поддержку.

Минусы:

- Ограниченность графики:

SFML не предназначен для создания сложных 3D-графических сцен.

- Нет встроенной поддержки GUI:

SFML не предоставляет готовые компоненты пользовательского интерфейса (GUI), как, например, кнопки, текстовые поля. Для этого можно использовать дополнительные библиотеки, или реализовать GUI самостоятельно.

- Более медленная работа, чем низкоуровневые библиотеки:

В сравнении с библиотеками, такими как OpenGL или Vulkan, SFML может быть немного медленнее, особенно при работе с очень большими объемами данных.

- Не подходит для крупных проектов:

SFML может быть не лучшим выбором для крупных проектов с сложной логикой и графикой, где требуется высокая производительность и гибкость.

2 Структура игры

2.1 Общее описание

Проект можно разбить на 3 типа классов: основные классы, в которых описываются объекты, их свойства и функции, классы «контроллеры», которые отвечают за управление другими классами и классы «менеджеры», которые отвечают за обработку данных из файла и генерацию классов на их основе. (См. Рисунок А.1).

Основной алгоритм игры:

1. В основном файле Spaceroguelike создаются окна игры, и в них добавляются элементы интерфейса, такие как кнопки, заголовки и полосы прогресса, после чего выводится главное меню.
2. При нажатии игроком кнопки старт инициализируется главный игровой контроллер Game_controller, отвечающий за управление игрой.
3. Game_controller инициализирует хранящиеся внутри классы и начинает их обработку.
 - Класс Entity_controller отвечающий за управление сущностями в целом.
 - Класс Enemy_controller, отвечающий за управление врагами.
 - Класс Shot_controller, отвечающий за управление снарядами.
 - Класс Wave_controller, отвечающий за генерацию волн врагов.
 - Классы менеджеры, отвечающие за обработку файлов и служащих для генерации классов на основе их прототипов.
 - Классы обработки текстур, звукового сопровождения и анимаций.
4. После инициализации класса Game_controller его методы используются в основном файле Spaceroguelike.
5. При окончании игры или её перезапуске Game_controller очищает свои данные и данные других классов.

2.2 Подробное описание элементов игры

2.2.1 Класс Entity

Класс содержит информацию о сущности: позицию, скорость, угол и скорость поворота, команду, номер слоя на котором будет отрисован, уникальный id, маяк, а также указатели на контроллеры текстур и звука.

Основные методы класса: `virtual void take_damage(Shot_type shot)` – отвечает за получение урона сущностью, `virtual void draw(sf::RenderWindow& window, bool is_show_hitboxes)` – отвечает за отрисовку сущности, `virtual void move(int time_passed)` – отвечает за передвижение сущности и обновление её модулей каждый кадр, `virtual float calculate_kill_value()` – отвечает за подсчёт стоимости сущности, используется при уничтожении противников, а также методы типа `get` и `set`, которые отвечают за внешнее управление классом (см. листинг A.1).

2.2.2 Класс Planet

Класс содержит информация о текстуре планеты, её хитбоксах, размерах и здоровье. Planet является наследником Entity.

Основные методы класса: методы типа `get` и `set`, которые отвечают за внешнее управление классом, а также перегруженные методы для передвижения и отрисовки описанные в пункте 2.2.1 (см. листинг A.2).

2.2.3 Класс Ship

Класс содержит контейнер `map` для хранения модулей корабля по имени, указатель на Entity, который определяет цель атаки, модуль корпуса корабля `Hull`, который отвечает за здоровье корабля.

Основные методы класса: `void init()` – отвечает за установку модуля корпуса корабля, `float turn_to_pos(sf::Vector2f target_pos, int time_passed)` – отвечает за разворот корабля в сторону указанных координат, `void input(sf::Vector2f a_control, bool a_is_move_demping, bool a_is_turn_demping, bool a_is_shooting)` – отвечает за управление модулями корабля, `void upgrade(Upgrade_info upgrade_info)` – реализует улучшение характеристик корабля по указанным параметрам, `Ship& add_unit(std::string name, Unit* unit)`

– отвечает за добавление модулей в корабль, методы типа `get` и `set`, которые отвечают за внешнее управление классом, а также перегруженные методы для передвижения и отрисовки описанные в пункте 2.2.1 (см. листинг А.3).

2.2.4 Класс Unit и его наследники

`Unit` – абстрактный класс, описывающий функции, реализуемые в наследниках.

Основные функции: `virtual void input(sf::Vector2f a_control, bool a_is_move_demping, bool a_is_turn_demping, bool a_is_shooting) = 0` – используется в наследниках для управления кораблём, `virtual void update(int time_passed) = 0` – чистая виртуальная функция, которая вызывается каждый кадр игрового цикла, `virtual void upgrade(Upgrade_info upgrade_info) = 0` – функция, которая позволяет улучшить характеристики юнита корабля, `virtual void upgrade(Upgrade_info upgrade_info) = 0` – функция, которая позволяет улучшить характеристики юнита корабля, `virtual void draw(sf::RenderWindow& window, bool is_show_hitboxes) override` – функция, которая отвечает отрисовку модулей (см. листинг А.4.1).

`Weaponry` – родительский класс, описывающий функции модулей оружия (см. листинг А.4.2).

`Plasma_gun` – класс, содержащий параметры оружия, такие как урон, скорость атаки, скорость полёта снаряда и дальность полёта снаряда (см. листинг А.4.3).

Функция `update` отвечает за выстрелы из оружия.

`Movement` – родительский класс, описывающий функции модулей, отвечающих за передвижение (см. листинг А.4.4).

`Engine` – класс, содержащий параметры двигателя, такие как ускорение и максимальная скорость (см. листинг А.4.5).

Функция `update` отвечает за движение корабля под заданным углом.

`RCS` – класс, содержащий параметры системы поворотов, такие как ускорение поворота и максимальная скорость поворота (см. листинг А.4.6).

Функция `update` отвечает за разворот корабля.

Demping – класс, содержащий параметры системы останова, такие как замедление при повороте и замедление при движении (см. листинг A.4.7).

Функция update отвечает за замедление корабля.

2.2.5 Класс Entity_controller

Класс содержит контейнер multiset для хранения указателей на сущность Entity, вектор из указателей на улучшения, указатели на команды игрока и врагов, указатели на корабль игрока и Землю, указатели на контроллеры, менеджер и интерфейс необходимые для внутренней работы класса.

Основные функции: void add_entity(Entity* entity) – отвечает за добавление новых сущностей в multiset entities и класс enemy_controller, void change_player_health_bar() – отвечает за обновление полосы здоровья игрока, void check_colisions() – отвечает за проверку коллизий сущностей, std::vector<Entity*> check_deaths(float& kill_val) – отвечает за проверку и обработку смертей сущностей, void reset() – отвечает за очищение данных в Entity_controller и Enemy_controller, методы типа get и set, которые отвечают за внешнее управление классом (см. листинг A.5).

2.2.6 Класс Enemy_controller

Класс содержит вектор из указателей на сущность Ship в котором хранится список враждебных к игроку кораблей enemies, вектор из указателей на сущность Entity, в котором хранятся сущности к которым враждебны корабли из прошлого списка, указатель на команду враждебных к игроку кораблей и указатель на корабль игрока.

Основные функции: void add_enemy(Ship* entity) и void add_entity(Entity* entity) – отвечают за добавление новых элементов в вектора, void remove_enemy(int id) и void remove_entity(int id) - отвечают за удаление элементов из векторов по их id, void control_enemy(int time_passed) - реализует искусственный интеллект врага, методы типа get и set, которые отвечают за внешнее управление классом (см. листинг A.6).

2.2.7 Класс Wave

Содержит параметры волны врагов, а именно: время между волнами, текущий прогресс волны, финальное значение волны, множители прогресса от времени и убийств, так же в классе хранится имя следующей волны, указатель на полосу прогресса и 3 вектора с событиями, прошедшими, текущими и будущими.

Основные функции: `void add_event(Wave_event* new_event)` – отвечает за добавление события в волну, `void kill_value(float value)` – отвечает за увеличение прогресса волны за убийства, `void update(int time_passed)` – отвечает за увеличение прогресса волны за время и обработку событий, методы типа `get` и `set`, которые отвечают за внешнее управление классом (см. листинг A.7)

2.2.8 Класс Wave_event

Содержит информацию о событии в волне, а именно: продолжительное ли событие, время начала и конца события, частоту появления врагов, координаты, где происходит событие, название события, контейнер `set` из маяков, и указатели на контроллеры, необходимые для внутренней работы.

Основные функции: `virtual void start()` – реализует работу мгновенного события, `virtual void update(int time_passed)` – реализует работу продолжительного события, методы типа `get` и `set`, которые отвечают за внешнее управление классом (см. листинг A.8).

2.2.9 Класс Wave_event_spawn_enemy

Содержит контейнер `map` для хранения кораблей по имени, вектор из указателей на `Ship_spawn`, в которых содержится информация о том, как создавать корабли и указатель на менеджер кораблей, нужный для внутренней работы. `Wave_event_spawn_enemy` - наследник `Wave_event`.

Основные функции: `void add_ship_type(std::string name, Ship* ship)` – отвечает за добавление кораблей в `map`, `void add_ship_spawn(Ship_spawn* ship_spawn)` – отвечает за добавление информации о том, как создавать

корабли в вектор, а также перегруженные методы описанные в пункте 2.2.8 (см. листинг А.9).

2.2.10 Класс Wave_event_continuous_spawn_enemy

Содержит контейнер map для хранения кораблей по имени, вектор из указателей на Ship_spawn, в которых содержится информация о том, как создавать корабли и указатель на менеджер кораблей, нужный для внутренней работы. Wave_event_continuous_spawn_enemy – наследник Wave_event.

Основные функции: void add_ship_type(std::string name, Ship* ship) – отвечает за добавление кораблей в map, void add_ship_spawn(Ship_spawn* ship_spawn) – отвечает за добавление информации о том, как создавать корабли в вектор, а также перегруженные методы описанные в пункте 2.2.8 (см. листинг А.10).

2.2.11 Структура Ship_spawn

Содержит информацию о том, как создавать корабли, а именно: шанс на появление дополнительного корабля, минимальная и максимальная дистанции на которой создавать врагов, минимальное и максимальное количество врагов которое может появиться, имя корабля, имена возможных улучшений, которые могут выпасть с убитого врага и шанс их выпадения.

2.2.12 Класс Wave_controller

Содержит указатель на текущую волну, указатель на команду врагов, указатель на полосу прогресса, а также контроллеры и менеджеры необходимые для внутренней работы. Основные функции: void init() – инициализирует первую волну, void reset() – очищает текущую волну, void generate_wave(std::string wave_name) – отвечает за генерацию волн по имени, void kill_value(float value) – отвечает за увеличение прогресса текущей волны за убийства, void update(int time_passed) – отвечает за генерацию новой волны в случае если старая кончилась (см. листинг А.11).

2.2.13 Класс Game_controller

Содержит команды игрока и противников, указатели, на корабль игрока и Землю, спрайт космоса, контроллеры и менеджеры нужные для внутренней

работы. Основные функции: `void init()` – отвечает за инициализацию основных элементов игры, `void reset()` – отвечает за сброс параметров игры, `void create_player()`, `void create_enemy(Ship* ship = nullptr)`, `void create_background()` и `void create_earth()` – отвечают за создание игрока, врага, заднего фона и Земли соответственно, `void change_player_health_bar();` и `void change_earth_health_bar()` – отвечают за изменение полосы здоровья игрока и Земли соответственно, `float player_turn_to_pos(sf::Vector2f target_pos, int time_passed)` – отвечает за разворот корабля игрока в сторону указанных координат, `void player_input(sf::Vector2f a_control, bool a_is_move_demping, bool a_is_turn_demping, bool a_is_shooting)` – используется для управления кораблём игрока, `bool update(int time_passed)` – отвечает за вызов функции `update` у `Wave_controller`, `void move(int time_passed)` – отвечает за вызов функции `move` у сущностей и выстрелов, а также отвечает за перемещение заднего фона, `void check_colisions()` – отвечает за вызов функции `check_colisions` у `Entity_controller`, `int check_deaths()` – отвечает за вызов функции `check_deaths` у `Entity_controller`, а также дополнительно обрабатывает смерть игрока и Земли, `void draw()` – отвечает за вызов функций отрисовки у всех сущностей и выстрелов, а также вызывает функции обновления полосок здоровья игрока и Земли (см. листинг А.12).

2.2.14 Основной файл `Spaceroguelike.cpp`

Содержит указатели на пользовательский интерфейс и игровой контроллер.

В основном файле, создаётся несколько объектов интерфейса типа `Screen`, на каждый экран добавляются соответствующие ему элементы, например на начальный экран “menu” добавляется заголовок с названием игры типа `Label`, а также кнопки типа `Button`: “start”, “continue”, “settings” и “exit”.

В `Spaceroguelike` также реализовано изменение масштаба по колёсику мыши, обработка кнопок клавиатуры и мыши, отслеживание состояния, в котором находится игра и вызов отрисовки всех элементов игры.

2.3 Вспомогательные алгоритмы

2.3.1 Алгоритмы чтения данных из файла, система прототипов и менеджеров

Для игры была реализована система получения данных из файла с расширением .cfg, она состоит из 3 элементов: генерируемого класса, его прототипа и менеджера.

Генерируемым классом может быть любой класс, для которого был реализован его прототип и менеджер.

Прототип – класс, созданный для хранения прочитанных менеджером данных из файла и генерации желаемого класса на их основе.

Менеджер – класс, отвечающий за чтение данных и составление контейнера map по имени из прототипов.

Для игры была реализована система получения данных из файла с расширением .cfg

Подробнее на примере класса Upgrade:

В конструкторе Upgrade_manager вызывается метод, который из файла “Upgrade_list.cfg” получает список файлов, которые нужно обработать, после чего метод void load_prototype(std::string file_name); создаёт контейнер типа map по имени из прототипов класса Upgrade – Upgrade_prototype.

В программе вызывается метод Upgrade* generate(std::string name, sf::Vector2f pos); который вызывает аналогичный метод у Upgrade_prototype.

Upgrade_prototype на основе имеющихся данных генерирует и возвращает объект класса Upgrade.

Пример файла “Attack_speed_upgrade.cfg” указан в листинге 3.1

Листинг 3.1 - Пример файла “Attack_speed_upgrade.cfg”

```
set : name = "Attack_speed";
set : texture = "Upgrades";
set : texture_rect = {1, 1, 32, 32}
add : rarity = {common, 200}
add : rarity = {uncommon, 100}
add : rarity = {rare, 20}
add : rarity = {epic, 5}
add : rarity = {legendary, 1}
add : property = {
    unit = "weapon1";
    name = "attack_time";
    amount = 10;
    flat = true;
}
```

Процесс обработки файла:

1. Вызывается функция `parse_file`, результатом работы которой является двумерный вектор из `std::string`, разбитый на токены. Токены – отдельные слова, последовательность слов, заключённые в двойные кавычки, либо специальные символы: “;” “,” “:” “=” “{” “}”. Также есть возможность ставить комментарии, которые будут проигнорированы при разбиении на токены, с помощью “//”.

2. Вызывается функция `parse_data`, которая принимает двумерный вектор из `std::string` и её результат работы – вектор из структур `File_command`. Правила построения `File_command`: в начале заполняется левая часть, элементы которой разбиваются двоеточием или запятой, после следует токен равенства и либо один токен с данными, либо фигурные скобки, внутри которых происходит рекурсивный вызов `parse_data` для разбиения на команды правой части. В случае ошибки вся команда игнорируется, а в консоль выводится предупреждение об ошибке.

3. Класс менеджера обрабатывает вектор из `File_command`, собирая данные по нужному правилу.

Такая система увеличивает масштабируемость и позволяет легко модифицировать любые части игры в которых задействована.

2.3.2 Создание текстур, анимации и звуков.

Для графической и аудио составляющей игры была использована библиотека SFML.

Библиотека позволяет делать графику при помощи текстур и спрайтов. Спрайты создаются внутри игры, а текстуры считываются из файла с помощью `Texture_controller`, в котором реализована функция, которая позволяет получать необходимую текстуру по её названию – `sf::Texture* get_texture(std::string name);`

Для звукового сопровождения был реализован аналогичный контроллер – `Sound_controller` с функцией `bool load_sound(std::string name, int count = 10, std::string path = "")`; Функция не только позволяет загружать звуки из файла, но и настраивает их звучание.

Для того чтобы создать анимации был реализован `Animation_controller`, который содержит вектор из анимаций, менеджер и указатели на контроллеры для внутренней работы.

Основные функции: `void create_animation(sf::Vector2f pos, sf::Vector2f size, std::string name)` – отвечает за создание анимации по имени, `void update(int time_passed)` – отвечает за постоянное изменение кадров анимации, `void draw()` – отвечает за отрисовку кадров анимации.

(См. листинг А.13)

2.3.3 Создание пользовательского интерфейса

Для игры был разработан класс `Item_holder`, в котором хранится контейнер типа `map` в котором находятся элементы интерфейса по имени.

Элементы интерфейса включают в себя:

- Кнопку `Button`
- Заголовок `Label`
- Полосу прогресса `Progress bar`

Для хранения этих элементов был разработан `Screen` – класс наследник от `Item_holder`, а для хранения экранов – `User_interface`, также наследник от `Item_holder`.

2.3.4 Вспомогательные функции

В результате разработки игры был создан файл с вспомогательными функциями – `Utility`. В нём также хранятся структура с информацией об улучшениях и структура с командами из файла.

Основные функции: `std::vector<std::vector<std::string>>`
`parse_file(std::string file_name)` – считывает файл, разбивает его на токены и возвращает данные для дальнейшей обработки, `std::vector<File_command>`
`parse_data(std::vector<std::vector<std::string>>& data, int start_line = -1, int start_token = -1, int end_line = -1, int end_token = -1)` – преобразовывает данные, возвращаемые предыдущим методом в структуру `File_command`, методы типа `get_data` – обрабатывают данные считанные `parse_file` и получают из них данные необходимого типа.

ЗАКЛЮЧЕНИЕ

По результатам работы была успешно разработана игра в жанре vampire-like про бои на космических кораблях и защиту Земли от уничтожения.

Также был разработан собственный игровой движок, и система загрузки данных из файла, что увеличивает масштабируемость и облегчает моддинг.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В пояснительной записке применяются следующие сокращения и обозначения:

БР – бакалаврская работа

ВС – вычислительная система

ПЗ – пояснительная записка

СибГУТИ – Сибирский государственный университет
телекоммуникаций и информатики

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Подбельский, В.В. Фомин С.С. Программирование на языке Си : Учеб. пособие – 2-е доп. изд. – М.: Финансы и статистика, 2001. – 600 с.
- 2 Лебедеенко, Л. Ф. Основы программирования на C++ : учебное пособие / Л. Ф. Лебедеенко, О. И. Моренкова ; Сибирский государственный университет телекоммуникаций и информатики. - Изд. 2-е . - Новосибирск : СибГУТИ.
- 3 Разработка игры на C++/SFML: Начало. URL: <https://habr.com/ru/articles/800691/>
- 4 Артур Морейра, Ян Халлер, Хенрик Фогелиус Ханссон: «Разработка игр с использованием SFML», 2013. 296с.
- 5 Майкл Доусон: «Программирование игр на C++», 2022. 352с.

ПРИЛОЖЕНИЕ А

Структура и код программы

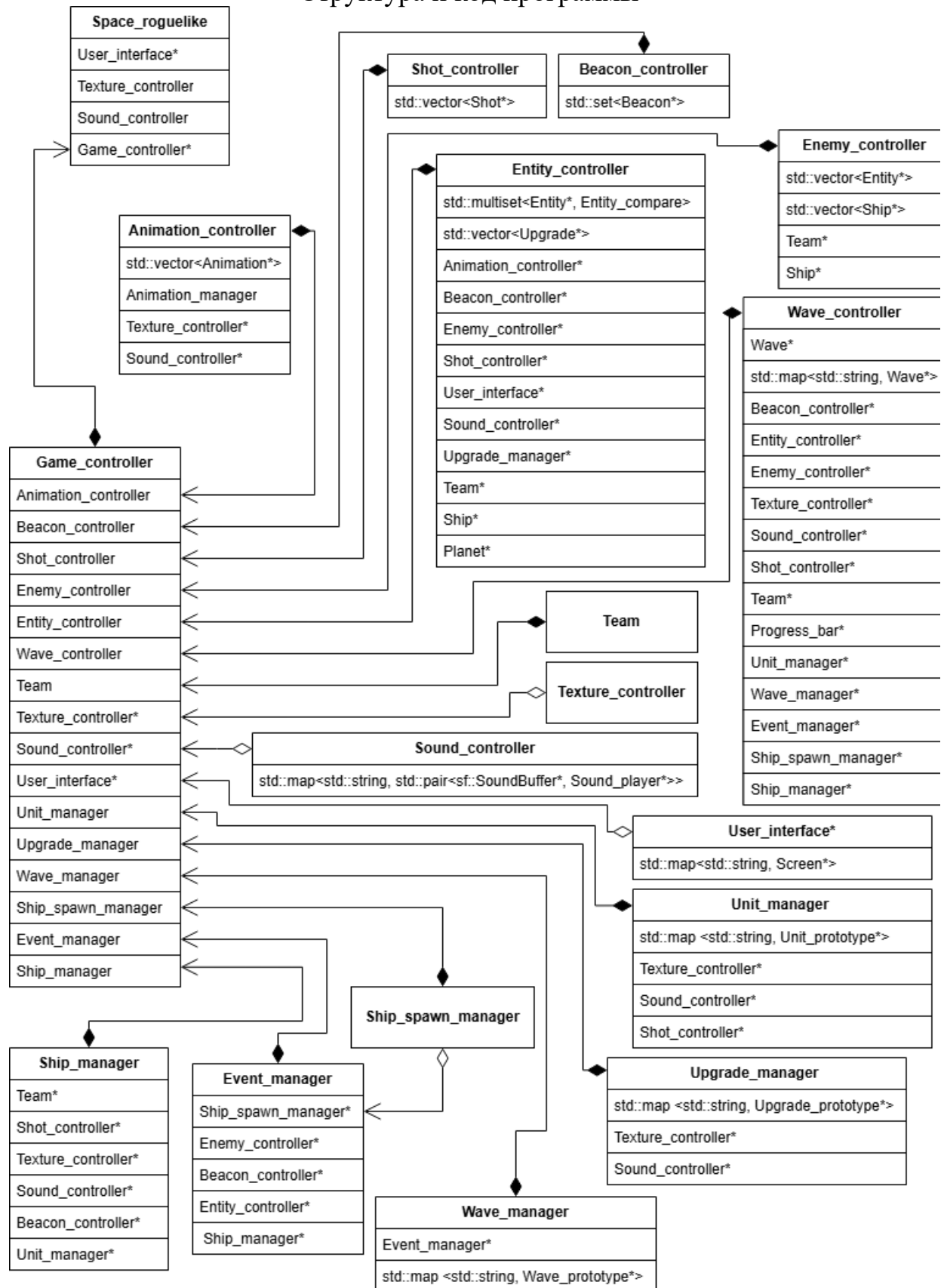


Рисунок А.1 – Схема программы

Листинг A.1 — Класс Entity

```
#include <SFML/Graphics.hpp>
#include "Shot_controller.h"
#include "Sound_controller.h"
#include "Texture_controller.h"
#include "Beacon.h"
#include "Shot_type.h"
#include "Hitbox.h"
#include "Team.h"

class Entity
{
private:
    static inline int last_id = 0;
protected:
    int id;
    bool is_planet;
    Team* team;
    sf::Vector2f position;
    sf::Vector2f velocity;
    float turn_speed;
    float turn_angle;
    int layer;
    Beacon* beacon;
    Sound_controller* sound_controller;
    Texture_controller* texture_controller;
public:
    Entity();
    Entity(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller, Beacon* a_beacon = nullptr, int
a_layer = 0);
    Entity(const Entity& copy, Beacon* a_beacon =
nullptr);
    ~Entity();
    virtual void take_damage(Shot_type shot);
    virtual void draw(sf::RenderWindow& window,
bool is_show_hitboxes);
    virtual void move(int time_passed);
    void set_velocity(sf::Vector2f new_velocity);
    void add_velocity(sf::Vector2f
additional_velocity);
    void set_turn_speed(float new_turn_speed);
    void add_turn_speed(float
additional_turn_speed);
    Entity* set_pos(sf::Vector2f new_position);
```

Листинг A.2 — Класс Planet

```
#include <SFML/Graphics.hpp>
#include "Sound_controller.h"
#include "Texture_controller.h"
#include "Entity.h"
#include "Hitbox.h"
#include "Team.h"

class Planet : public Entity
{
protected:
    sf::Texture* texture;
    sf::CircleShape planet_sprite;
    sf::CircleShape hitbox;
    std::vector<Hitbox> hitboxes;
    float cur_health;
    float max_health;
    float radius;
public:
    Planet(Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller, Team* a_team,
Beacon* a_beacon = nullptr, int a_layer = 0);
    ~Planet();
    Planet* set_position(sf::Vector2f pos);
    Planet* set_cur_health(float val);
    Planet* set_max_health(float val);
    Planet* set_radius(float val);
    virtual std::vector<Hitbox> get_hitboxes()
override;
    virtual float get_current_health() override;
    virtual float get_max_health() override;
    virtual float get_size() override;
    virtual void take_damage(Shot_type shot);
    virtual void move(int time_passed) override;
    virtual void draw(sf::RenderWindow& window, bool
is_show_hitboxes) override;
};
```

Листинг A.3 — Класс Ship

```
#include <SFML/Graphics.hpp>
#include <math.h>
#include "Entity.h"
#include "Shot_controller.h"
#include "Shot.h"
#include "Shot_type.h"
#include "Plasma_shot.h"
#include "Hitbox.h"
#include "Progress_bar.h"
#include "Unit.h"
```

```

#include "Utility.h"
#include "Weaponry.h"
#include "Plasma_gun.h"
#include "Movement.h"
#include "Engine.h"
#include "RCS.h"
#include "Demping.h"
#include "Hull.h"
#include "Beacon_controller.h"
#include "Unit_manager.h"

class Ship : public Entity
{
protected:
    Shot_controller* shot_controller;
    std::map<std::string, Unit*> units;
    Entity* target;
    Hull* hull_unit;
public:
    Ship(sf::Vector2f a_position, float
a_turn_angle, Team* a_team, Shot_controller*
a_shot_controller, Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller,
        Beacon* a_beacon = nullptr, int a_layer = 0);
    Ship(const Ship& copy, Beacon* a_beacon =
nullptr);
    ~Ship();
    virtual void take_damage(Shot_type shot)
override;
    virtual void draw(sf::RenderWindow& window,
bool is_show_hitboxes) override;
    virtual void move(int time_passed) override;
    virtual std::vector<Hitbox> get_hitboxes()
override;
    virtual float get_current_health() override;
    virtual float get_max_health() override;
    virtual float get_size() override;
    Entity* get_target();
    float get_stat_val(std::string unit_name,
std::string property_name);
    void set_target(Entity* new_target = nullptr);
    void remove_target(Entity* a_target);
    virtual float calculate_kill_value() override;
    void init();
    float turn_to_pos(sf::Vector2f target_pos, int
time_passed);
    void acselerate(float speed_change, int
time_passed);
    void input(sf::Vector2f a_control, bool
a_is_move_demping, bool a_is_turn_demping, bool
a_is_shooting);

```

```

        void upgrade(Upgrade_info upgrade_info);
        Ship& add_unit(std::string name, Unit* unit);

};
class Ship_prototype
{
protected:
    std::vector<std::pair<std::string, float>>
units;
    Team* enemy_team;
    Shot_controller* shot_controller;
    Texture_controller* texture_controller;
    Sound_controller* sound_controller;
    Beacon_controller* beacon_controller;
    Unit_manager* unit_manager;

public:
    Ship_prototype(Team* a_enemy_team,
Shot_controller* a_shot_controller,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller,
Beacon_controller* a_beacon_controller,
    Unit_manager* a_unit_manager);
    ~Ship_prototype();
    void add_unit(std::vector<File_command>&
commands);

    virtual Ship* generate();
};

```

Листинг A.4.1 — Класс Unit

```

#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <map>
#include <string>
#include "Utility.h"
#include "Sound_controller.h"
#include "Texture_controller.h"
#include "Team.h"
#include "Hitbox.h"
enum Prototype_parametr_types {
    Prototype_parametr_none,
    Prototype_parametr_const_mod,
    Prototype_parametr_min_mod,
    Prototype_parametr_max_mod,
    Prototype_parametr_val_multiplier,
    Prototype_parametr_unknown = -1
};

```

```

enum Unit_type {
    unit_none,
    unit_plasma_gun,
    unit_engine,
    unit_rcs,
    unit_demping,
    unit_hull,
    unit_unknown = -1
};

struct Prototype_params {
    std::string name;
    float const_mod;
    float min_mod;
    float max_mod;
    float val_multiplier;
    std::map<float, float> const_line;
    std::map<float, float> min_line;
    std::map<float, float> max_line;
    void set_param(std::vector<std::string>& data);
    float generate(float danger_lvl);
private:
    float calc_mod(std::map<float, float>& test,
float danger_lvl);
};

class Ship;

class Unit
{
protected:
    bool is_visible;
    sf::Vector2f pos;
    sf::Vector2f global_pos;
    float angle;
    float global_angle;
    Team* team;
    Ship* ship;
    sf::Texture* texture = nullptr;
    sf::RectangleShape* sprite_rect = nullptr;
    Sound_controller* sound_controller;
    Texture_controller* texture_controller;
public:
    Unit(bool a_is_visible, sf::Vector2f a_pos, float
a_angle, Ship* a_ship, Team* a_team,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller);
    Unit(const Unit& copy);
    virtual Unit* copy() = 0;
    virtual void draw(sf::RenderWindow& window, bool
is_show_hitboxes) = 0;

```

```

        virtual void update(int time_passed) = 0;
        virtual void input(sf::Vector2f a_control, bool
a_is_move_demping, bool a_is_turn_demping, bool
a_is_shooting) = 0;
        virtual std::vector<Hitbox> get_hitboxes() = 0;
        virtual float calculate_kill_value();
        virtual std::string get_type_name();
        virtual float get_stat_val(std::string
property_name) = 0;
        void set_ship(Ship* new_ship);
        void update_pos(sf::Vector2f new_pos, float
new_angle);
        virtual void upgrade(Upgrade_info upgrade_info) =
0;
};

class Unit_prototype {
protected:
    Unit_type type;
    Sound_controller* sound_controller;
    Texture_controller* texture_controller;
public:
    Unit_prototype(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller);
    ~Unit_prototype();
    void set_unit_type(Unit_type new_type);
    Unit_type get_unit_type();
    virtual Unit* generate(Ship* ship, Team* team,
float danger_lvl) = 0;
    virtual Unit_prototype*
set_param(std::vector<std::string>& data) = 0;
};

```

Листинг A.4.2 — Класс Weaponry

```

#include "Unit.h"
#include "Shot_controller.h"

class Weaponry : public Unit
{
protected:
    bool is_shooting;
public:
    Weaponry(bool a_is_visible, sf::Vector2f a_pos,
float a_angle, Ship* a_ship, Team* a_team,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller);
    Weaponry(const Weaponry& copy);
    virtual Unit* copy() override;
    virtual void draw(sf::RenderWindow& window, bool
is_show_hitboxes) override;
    virtual void update(int time_passed) override;
};

```

```

        virtual void input(sf::Vector2f a_control, bool
a_is_move_demping, bool a_is_turn_demping, bool
a_is_shooting) override;
        virtual std::vector<Hitbox> get_hitboxes() override;
        virtual void upgrade(Upgrade_info upgrade_info)
override;
        virtual float get_stat_val(std::string
property_name) override;
};

class Weaponry_prototype : public Unit_prototype{
public:
    Weaponry_prototype(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller);
    ~Weaponry_prototype();

    virtual Unit* generate(Ship* ship, Team* team,
float danger_lvl) = 0;
};

```

Листинг A.4.3— Класс Plasma_gun

```

#include "Weaponry.h"
#include "Plasma_shot.h"
#include "Shot_controller.h"
#include "Utility.h"
class Plasma_gun : public Weaponry
{
protected:
    float shot_speed = 500.0;
    float shot_travel_time = 1500;
    float shot_damage = 10;
    int attack_delay = 0;
    int attack_time = 500;
    Shot_controller* shot_controller;
    float shot_speed_val_multiplier = 0.001f;
    float shot_travel_time_val_multiplier = 0.0005f;
    float shot_damage_val_multiplier = 0.5f;
    float attack_time_val_multiplier = 1.f;
public:
    Plasma_gun(bool a_is_visible, sf::Vector2f a_pos,
float a_angle, Ship* a_ship, Team* a_team,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller,
Shot_controller* a_shot_controller);
    Plasma_gun(const Plasma_gun& copy);
    virtual Unit* copy() override;
    virtual void draw(sf::RenderWindow& window, bool
is_show_hitboxes) override;
    virtual void update(int time_passed) override;
    virtual std::vector<Hitbox> get_hitboxes()
override;
    virtual std::string get_type_name() override;
};

```

```

        virtual float get_stat_val(std::string
property_name) override;
        virtual float calculate_kill_value() override;
        Plasma_gun* set_shot_speed(float new_shot_speed);
        Plasma_gun* set_shot_travel_time(float
new_shot_travel_time);
        Plasma_gun* set_shot_damage(float
new_shot_damage);
        Plasma_gun* set_attack_time(float
new_attack_time);
        Plasma_gun* set_shot_speed_val_multiplier(float
new_shot_speed_val_multiplier);
        Plasma_gun* set_shot_damage_val_multiplier(float
new_shot_damage_val_multiplier);
        Plasma_gun* set_attack_time_val_multiplier(float
new_attack_time_val_multiplier);
        virtual void upgrade(Upgrade_info upgrade_info)
override;
};
class Plasma_gun_prototype : public
Weaponry_prototype {
protected:
        Prototype_params shot_speed;
        Prototype_params shot_distance;
        Prototype_params shot_damage;
        Prototype_params attack_speed;
        Shot_controller* shot_controller;
public:
        Plasma_gun_prototype(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller, Shot_controller*
a_shot_controller);
        ~Plasma_gun_prototype();
        virtual Unit* generate(Ship* ship, Team* team,
float danger_lvl);
        virtual Unit_prototype*
set_param(std::vector<std::string>& data) override;
};

```


Листинг A.4.4— Класс Movement

```
#include "Unit.h"
class Movement : public Unit
{
protected:
    sf::Vector2f control;
    bool is_move_demping;
    bool is_turn_demping;
public:
    Movement(bool a_is_visible, sf::Vector2f a_pos,
float a_angle, Ship* a_ship, Team* a_team,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller);
    Movement(const Movement& copy);
    virtual Unit* copy() override;
virtual void draw(sf::RenderWindow& window, bool
is_show_hitboxes) override;
    virtual void input(sf::Vector2f a_control, bool
a_is_move_demping, bool a_is_turn_demping, bool
a_is_shooting) override;
    virtual void update(int time_passed) override;
    virtual std::vector<Hitbox> get_hitboxes()
override;
    virtual void upgrade(Upgrade_info upgrade_info)
override;
    virtual float get_stat_val(std::string
property_name) override;
};

class Movement_prototype : public Unit_prototype {
public:
    Movement_prototype(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller);
    ~Movement_prototype();
    virtual Unit* generate(Ship* ship, Team* team,
float danger_lvl) = 0;
};
```

Листинг A.4.5— Класс Engine

```
#define USE_MATH_DEFINES
#include <Math.h>
#include "Movement.h"
class Engine : public Movement
{
protected:
    float acceleration_speed;
    float speed_limit;
    const float acceleration_speed_val_multiplier =
0.005f;
    const float speed_limit_val_multiplier = 0.001f;
public:
    Engine(bool a_is_visible, sf::Vector2f a_pos,
float a_angle, Ship* a_ship, Team* a_team,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller);
    Engine(const Engine& copy);
    virtual Unit* copy() override;
virtual void draw(sf::RenderWindow& window, bool
is_show_hitboxes) override;
    virtual void update(int time_passed) override;
    virtual std::vector<Hitbox> get_hitboxes()
override;
    virtual std::string get_type_name() override;
    virtual float get_stat_val(std::string
property_name) override;
    virtual float calculate_kill_value() override;
    Engine* set_acceleration_speed(float val);
    Engine* set_speed_limit(float val);
    virtual void upgrade(Upgrade_info upgrade_info)
override;
};

class Engine_prototype : public Movement_prototype {
protected:
    Prototype_params acceleration_speed;
    Prototype_params speed_limit;
public:
    Engine_prototype(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller);
    ~Engine_prototype();
    virtual Unit* generate(Ship* ship, Team* team,
float danger_lvl) override;
    virtual Unit_prototype*
set_param(std::vector<std::string>& data) override;
};
```

Листинг A.4.6— Класс RCS

```
#define USE_MATH_DEFINES
#include <Math.h>
#include "Movement.h"

class RCS : public Movement
{
protected:
    float turn_acceleration;
    float max_turn_speed;
    const float turn_acceleration_val_multiplier =
0.005f;
    const float max_turn_speed_val_multiplier =
0.001f;
public:
    RCS(bool a_is_visible, sf::Vector2f a_pos,
float a_angle, Ship* a_ship, Team* a_team,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller);
    RCS(const RCS& copy);
    virtual Unit* copy() override;
    virtual void draw(sf::RenderWindow& window,
bool is_show_hitboxes) override;
    virtual void update(int time_passed) override;
    virtual std::vector<Hitbox> get_hitboxes()
override;
    virtual std::string get_type_name() override;
    virtual float get_stat_val(std::string
property_name) override;
    virtual float calculate_kill_value() override;
    RCS* set_turn_acceleration(float val);
    RCS* set_max_turn_speed(float val);
    virtual void upgrade(Upgrade_info upgrade_info)
override;
};

class RCS_prototype : public Movement_prototype {
protected:
    Prototype_params turn_acceleration;
    Prototype_params max_turn_speed;

public:
    RCS_prototype(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller);
    ~RCS_prototype();
    virtual Unit* generate(Ship* ship, Team* team,
float danger_lvl) override;
    virtual Unit_prototype*
set_param(std::vector<std::string>& data) override;
};
```

Листинг A.4.7 — Класс Damping

```
#define USE_MATH_DEFINES
#include <Math.h>
#include "Movement.h"
class Damping : public Movement
{
protected:
    float move_damping_power;
    float turn_damping_power;
    const float move_damping_power_val_multiplier =
0.005f;
    const float turn_damping_power_val_multiplier =
0.005f;
public:
    Damping(bool a_is_visible, sf::Vector2f a_pos,
float a_angle, Ship* a_ship, Team* a_team,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller);
    Damping(const Damping& copy);
    virtual Unit* copy() override;
    virtual void draw(sf::RenderWindow& window, bool
is_show_hitboxes) override;
    virtual void update(int time_passed) override;
    virtual std::vector<Hitbox> get_hitboxes()
override;
    virtual std::string get_type_name() override;
    virtual float get_stat_val(std::string
property_name) override;
    virtual float calculate_kill_value() override;
    Damping* set_move_damping_power(float val);
    Damping* set_turn_damping_power(float val);
    virtual void upgrade(Upgrade_info upgrade_info)
override;
};
class Damping_prototype : public Movement_prototype {
protected:
    Prototype_params move_damping_power;
    Prototype_params turn_damping_power;
public:
    Damping_prototype(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller);
    ~Damping_prototype();
    virtual Unit* generate(Ship* ship, Team* team,
float danger_lvl) override;
    virtual Unit_prototype*
set_param(std::vector<std::string>& data) override;
};
```

Листинг А.4.8— Класс Hull

```
#include "Unit.h"
#include "Shot.h"
#include "Progress_bar.h"

class Hull : public Unit
{
protected:
    sf::RectangleShape sprite_rect;
    sf::RectangleShape hitbox_rect;
    sf::Texture* texture;
    std::vector<Hitbox> hitboxes;
    bool show_mini_health_bar = true;
    Progress_bar health_bar;
    float max_health_points;
    float cur_health_points;
    const float max_health_points_val_multiplier =
0.5f;
public:
    Hull(bool a_is_visible, sf::Vector2f a_pos, float
a_angle, Ship* a_ship, Team* a_team,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller, std::string
a_texture_name);
    Hull(const Hull& copy);
    virtual Unit* copy() override;
    virtual void draw(sf::RenderWindow& window, bool
is_show_hitboxes) override;
    virtual void update(int time_passed) override;
    virtual void input(sf::Vector2f a_control, bool
a_is_move_demping, bool a_is_turn_demping, bool
a_is_shooting) override;
    virtual std::vector<Hitbox> get_hitboxes()
override;
    sf::RectangleShape get_sprite_rect();
    sf::RectangleShape get_hitbox_rect();
    virtual std::string get_type_name() override;
    virtual float get_stat_val(std::string
property_name) override;
    float get_max_health_points();
    float get_cur_health_points();
    bool take_damage(Shot_type shot);
    virtual float calculate_kill_value() override;
    Hull* set_health_points(float val);
    virtual void upgrade(Upgrade_info upgrade_info)
override;
};
class Hull_prototype : public Unit_prototype {
protected:
    Prototype_params max_health_points;
```

```

public:
    Hull_prototype(Texture_controller*
a_texture_controller, Sound_controller*
a_sound_controller);
    ~Hull_prototype();
    virtual Unit* generate(Ship* ship, Team* team,
float danger_lvl) override;
    virtual Unit_prototype*
set_param(std::vector<std::string>& data) override;
};

```

Листинг A.5 — Класс Entity_controller

```

#include <SFML/Graphics.hpp>
#include <string>
#include <set>
#include "Animation_controller.h"
#include "Beacon_controller.h"
#include "Enemy_controller.h"
#include "Shot_controller.h"
#include "Sound_controller.h"
#include "Upgrade_manager.h"
#include "Upgrade.h"
#include "Entity.h"
#include "Ship.h"
#include "Planet.h"
#include "Team.h"
#include "User_interface.h"
class Entity_controller
{
private:
    std::multiset<Entity*, Entity_compare> entities;
    std::vector<Upgrade*> drops;
sf::RenderWindow* window;
    Animation_controller* animation_controller;
    Beacon_controller* beacon_controller;
    Enemy_controller* enemy_controller;
    Shot_controller* shot_controller;
    User_interface* user_interface;
    Sound_controller* sound_controller;
    Upgrade_manager* upgrade_manager;
Team* player_team;
    Team* enemy_team;
    Ship* player_ship = nullptr;
    Planet* earth = nullptr;
    bool is_show_hitboxes = false;

```

```

public:
    Entity_controller(sf::RenderWindow*
program_window, User_interface* a_user_interface,
Enemy_controller* a_enemy_controller,
Shot_controller* a_shot_controller,
    Sound_controller* a_sound_controller,
Animation_controller* a_animation_controller,
Beacon_controller* a_beacon_controller,
Upgrade_manager* a_upgrade_manager, Team*
a_player_team,
    Team* a_enemy_team);
    ~Entity_controller();

    void add_entity(Entity* entity);
    void player_input(sf::Vector2f a_control, bool
a_is_move_demping, bool a_is_turn_demping, bool
a_is_shooting);
    void change_player_health_bar();
    void hitboxes_visability(bool status);
    void hitboxes_visability_toggle();
    void enemy_force_shot(int time_passed);
    void check_colisions();
    std::vector<Entity*> check_deaths(float&
kill_val);
    void move(int time_passed);
    void set_player_ship(Ship* new_player_ship);
    void set_earth(Planet* new_earth);

    sf::Vector2f get_player_pos();
    sf::Vector2f get_earth_pos();

    float get_stat_val(std::string unit_name,
std::string property_name);

    void spawn_upgrade();

    void init();
    void reset();

    void draw();
};

```

Листинг A.6 — Класс Enemy_controller

```
#define USE_MATH_DEFINES
#include <SFML/Graphics.hpp>
#include <Math.h>
#include "Shot_controller.h"
#include "Entity.h"
#include "Ship.h"
#include "Team.h"
#include "Utility.h"

class Enemy_controller
{
private:
    std::vector<Entity*> entities;
    std::vector<Ship*> enemies;
    Team* enemy_team;
    Ship* player_ship = nullptr;
public:
    Enemy_controller(Team* a_enemy_team);
    ~Enemy_controller();
    void add_enemy(Ship* entity);
    void add_entity(Entity* entity);
    void remove_enemy(int id);
    void remove_entity(int id);
    void set_player_ship(Ship* new_player_ship);
    int get_enemy_count();
    void control_enemy(int time_passed);
    void force_shot(int time_passed);
    void reset();
};
```

Листинг A.7 — Класс Wave

```
#include <vector>
#include "Wave_event.h"
#include "Event_manager.h"
#include "Progress_bar.h"
#include "Wave_event_continuous_spawn_enemy.h"
class Wave
{
protected:
    int wave_delay;
    float wave_progress;
    float wave_end;
    float time_factor;
    float kill_factor;
    std::string next_wave_name;
```



```

        std::vector<Wave_event*> past_wave_events;
        std::vector<Wave_event*> current_wave_events;
        std::vector<Wave_event*> future_wave_events;
        Progress_bar* wave_progress_bar;
public:
    Wave(int a_wave_delay, float a_wave_end, float
a_time_factor, float a_kill_factor, std::string
a_next_wave_name, Progress_bar* a_wave_progress_bar);
    ~Wave();
    void add_event(Wave_event* new_event);
    void kill_value(float value);
    void update(int time_passed);
    void draw(sf::RenderWindow& window);
    float get_wave_end();
    std::string get_next_wave_name();
    bool is_over();
};

class Wave_prototype {
private:
    struct Event_prototype_info;
protected:
    Event_manager* event_manager;
    int wave_delay;
    float wave_progress;
    float wave_end;
    float time_factor;
    float kill_factor;
    std::string next_wave;
    std::vector<Event_prototype_info*> wave_events;
public:
    Wave_prototype(Event_manager* a_event_manager);
    ~Wave_prototype();

    void set_wave_delay(int val);
    void set_wave_proress(float val);
    void set_wave_end(float val);
    void set_time_factor(float val);
    void set_kill_factor(float val);
    void set_next_wave(std::string val);
    void add_wave_event(std::string wave_event_name,
float wave_event_time_start, float
wave_event_time_end);
    Wave* generate(Progress_bar* wave_progress_bar);
};

```

Листинг А.8— Класс Wave_event

```
#include <string>
#include <SFML/Graphics.hpp>
#include "Beacon_controller.h"
#include "Entity_controller.h"
#include "Enemy_controller.h"
#include "Ship_spawn_manager.h"

class Wave_event
{
protected:
    bool is_continious;
    float time_start;
    float time_end;
    float spawn_freq;
    sf::Vector2f pos;
    std::string name;
    std::set<Beacon*> beacons;
    Beacon_controller* beacon_controller;
    Entity_controller* entity_controller;
    Enemy_controller* enemy_controller;
public:
    Wave_event(bool a_is_continious, float
a_time_start, float a_time_end, sf::Vector2f a_pos,
std::string a_name, Entity_controller*
a_entity_controller, Enemy_controller*
a_enemy_controller, Beacon_controller*
a_beacon_controller);
    ~Wave_event();
    virtual void start();
    virtual void update(int time_passed);
    float get_time_start();
    float get_time_end();
    bool get_is_continious();
};

class Wave_event_prototype {
protected:
    bool is_continious;
    float spawn_freq;
    std::string origin;
    std::string name;
    Ship_spawn_manager* ship_spawn_manager;
public:
    Wave_event_prototype(Ship_spawn_manager*
a_ship_spawn_manager);
    ~Wave_event_prototype();
};
```

```

        void set_is_continuous(bool val);
        void set_spawn_freq(float val);
        void set_origin(std::string val);
        void set_name(std::string val);
        virtual void add_ship_spawn_name(std::string
val);
        virtual Wave_event* generate(float time_start,
float time_end, Entity_controller* entity_controller,
Enemy_controller* enemy_controller,
Beacon_controller* beacon_controller) = 0;
};

```

Листинг A.9 — Класс Wave_event_spawn_enemy

```

#include <string>
#include <map>
#include "Ship.h"
#include "Ship_spawn.h"
#include "Ship_manager.h"
#include "Ship_spawn_manager.h"
#include "Wave_event.h"
#include "Entity_controller.h"
class Wave_event_spawn_enemy : public Wave_event
{
protected:
    std::map<std::string, Ship*> ships;
    std::vector<Ship_spawn*> ship_spawns;
    Ship_manager* ship_manager;
public:
    Wave_event_spawn_enemy(float a_time_start,
sf::Vector2f a_pos, std::string a_name,
Entity_controller* a_entity_controller,
Enemy_controller* a_enemy_controller,
Beacon_controller* a_beacon_controller,
Ship_manager* a_ship_manager);
    ~Wave_event_spawn_enemy();
    virtual void start() override;
    void add_ship_type(std::string name, Ship* ship);
    void add_ship_spawn(Ship_spawn* ship_spawn);
};
class Wave_event_spawn_enemy_prototype : public
Wave_event_prototype {
protected:
    std::vector<std::string> ship_names;
    std::vector<std::string> ship_spawn_names;
    Ship_manager* ship_manager;
    Entity_controller* entity_controller;
public:
    Wave_event_spawn_enemy_prototype(Ship_manager*
a_ship_manager, Ship_spawn_manager*
a_ship_spawn_manager, Entity_controller*
a_entity_controller);

```

```

        ~Wave_event_spawn_enemy_prototype();
        void set_ship_names(std::vector<std::string>
val);
        void
set_ship_spawn_names(std::vector<std::string> val);
        virtual void add_ship_spawn_name(std::string val)
override;
        virtual Wave_event_spawn_enemy* generate(float
time_start, float time_end, Entity_controller*
entity_controller, Enemy_controller*
enemy_controller, Beacon_controller*
beacon_controller) override;
};

```

Листинг A.10— Класс Wave_event_continuous_spawn_enemy

```

#include "Ship.h"
#include "Ship_spawn.h"
#include "Ship_manager.h"
#include "Ship_spawn_manager.h"
#include "Wave_event.h"
class Wave_event_continuous_spawn_enemy : public
Wave_event
{
protected:
    std::map<std::string, Ship*> ships;
    std::vector<Ship_spawn*> ship_spawns;
    Ship_manager* ship_manager;
    float spawn_delay = 0;
public:
    Wave_event_continuous_spawn_enemy(float
a_time_start, float a_time_end, float a_spawn_freq,
sf::Vector2f a_pos, std::string a_name,
Entity_controller* a_entity_controller,
    Enemy_controller* a_enemy_controller,
Beacon_controller* a_beacon_controller, Ship_manager*
a_ship_manager);
    ~Wave_event_continuous_spawn_enemy();
    virtual void update(int time_passed) override;
    void add_ship_type(std::string name, Ship* ship);
    void add_ship_spawn(Ship_spawn* ship_spawn);
};
class Wave_event_continuous_spawn_enemy_prototype :
public Wave_event_prototype {
protected:
    std::vector<std::string> ship_names;
    std::vector<std::string> ship_spawn_names;
    Ship_manager* ship_manager;
    float spawn_delay = 0;

```

```

public:

Wave_event_continuous_spawn_enemy_prototype(Ship_manager*
a_ship_manager, Ship_spawn_manager*
a_ship_spawn_manager);
    virtual Wave_event_continuous_spawn_enemy*
generate(float time_start, float time_end,
Entity_controller* entity_controller, Enemy_controller*
enemy_controller, Beacon_controller* beacon_controller)
override;
};

~Wave_event_continuous_spawn_enemy_prototype();
void set_ship_names(std::vector<std::string> val);
void set_ship_spawn_names(std::vector<std::string>
val);
    virtual void add_ship_spawn_name(std::string val)
override;
    void set_spawn_freq(float val);

```

Листинг A.11— Класс Wave_controller

```

#include <algorithm>
#include <stdlib.h>
#include <string>
#include <map>
#include "Beacon_controller.h"
#include "Entity_controller.h"
#include "Enemy_controller.h"
#include "Texture_controller.h"
#include "Sound_controller.h"
#include "Shot_controller.h"
#include "Team.h"
#include "Wave.h"
#include "Wave_event.h"
#include "Wave_event_spawn_enemy.h"
#include "Wave_event_continuous_spawn_enemy.h"
#include "Progress_bar.h"
#include "Ship_spawn.h"
#include "Utility.h"
#include "Unit_manager.h"
#include "Wave_manager.h"
#include "Event_manager.h"
#include "Ship_spawn_manager.h"
#include "Ship_manager.h"

class Wave_controller
{
protected:
    Wave* cur_wave;
    Beacon_controller* beacon_controller;

```

```

Entity_controller* entity_controller;
Enemy_controller* enemy_controller;
Texture_controller* texture_controller;
Sound_controller* sound_controller;
Shot_controller* shot_controller;
Team* enemy_team;
Progress_bar* wave_progress_bar;
Unit_manager* unit_manager;
Wave_manager* wave_manager;
Event_manager* event_manager;
Ship_spawn_manager* ship_spawn_manager;
Ship_manager* ship_manager;

public:
    Wave_controller(Entity_controller*
a_entity_controller, Enemy_controller*
a_enemy_controller, Texture_controller*
a_texture_controller,
        Sound_controller* a_sound_controller,
Shot_controller* a_shot_controller,
Beacon_controller* a_beacon_controller, Team*
a_enemy_team, Progress_bar* a_wave_progress_bar,
        Unit_manager* a_unit_manager, Wave_manager*
a_wave_manager, Event_manager* a_event_manager,
Ship_spawn_manager* a_ship_spawn_manager,
Ship_manager* a_ship_manager);
    ~Wave_controller();
    void init();
    void reset();
    void generate_wave(std::string wave_name);
    void kill_value(float value);
    bool update(int time_passed);
};

```

Листинг A.12— Класс Game_controller

```

#define USE_MATH_DEFINES
#include <SFML/Graphics.hpp>
#include <string>
#include <stdlib.h>
#include <Math.h>
#include "Animation_controller.h"
#include "Beacon_controller.h"
#include "Entity_controller.h"
#include "Enemy_controller.h"
#include "Shot_controller.h"
#include "Texture_controller.h"
#include "Sound_controller.h"
#include "Wave_controller.h"

```

```

#include "Entity.h"
#include "Planet.h"
#include "Ship.h"
#include "Team.h"
#include "Wave.h"
#include "User_interface.h"
#include "Unit_manager.h"
#include "Upgrade_manager.h"
#include "Wave_manager.h"
#include "Event_manager.h"
#include "Ship_spawn_manager.h"
#include "Ship_manager.h"
#include "Button.h"
#include "Label.h"
#include "Text_edit.h"
#include "Progress_bar.h"
class Game_controller
{
private:
    Team player_team;
    Team enemy_team;
    Animation_controller animation_controller;
    Beacon_controller beacon_controller;
    Shot_controller shot_controller;
    Enemy_controller enemy_controller;
    Entity_controller entity_controller;
    Wave_controller wave_controller;
    Texture_controller* texture_controller;
    Sound_controller* sound_controller;
    User_interface* user_interface;
    Unit_manager unit_manager;
    Upgrade_manager upgrade_manager;
    Wave_manager wave_manager;
    Ship_spawn_manager ship_spawn_manager;
    Event_manager event_manager;
    Ship_manager ship_manager;
    sf::RenderWindow* window;
    sf::Sprite space_sprite;
    Ship* player_ship = nullptr;
    Planet* earth = nullptr;
    int spawn_timer = 0;
    bool is_game_running = false;
public:
    Game_controller(sf::RenderWindow* program_window,
        User_interface* a_user_interface, Texture_controller*
        a_texture_controller, Sound_controller*
        a_sound_controller);
    ~Game_controller();

```

```

void init();
void reset();
void create_player();
void create_enemy(Ship* ship = nullptr);
void create_background();
void create_earth();
void spawn_upgrade();
void change_player_health_bar();
void change_earth_health_bar();
float player_turn_to_pos(sf::Vector2f target_pos,
int time_passed);
void player_input(sf::Vector2f a_control, bool
a_is_move_demping, bool a_is_turn_demping, bool
a_is_shooting);
bool update(int time_passed);
void move(int time_passed);
void check_colisions();
int check_deaths();
void hitboxes_visability(bool status);
void hitboxes_visability_toggle();
sf::Vector2f get_player_pos();
bool get_is_game_running();
float get_stat_val(std::string unit_name,
std::string property_name);
void draw();
};

```

Листинг A.13— Класс Animation_controller

```

#include <SFML/Graphics.hpp>
#include <string>
#include <map>
#include "Animation.h"
#include "Animation_manager.h"
#include "Texture_controller.h"
#include "Sound_controller.h"

class Animation_controller
{
protected:
    std::vector<Animation*> animations;
    Animation_manager animation_manager;
    Texture_controller* texture_controller;
    Sound_controller* sound_controller;
    sf::RenderWindow* window;
public:
    Animation_controller(sf::RenderWindow* a_window,
Texture_controller* a_texture_controller,
Sound_controller* a_sound_controller);

```



```
~Animation_controller();  
    void init();  
    void reset();  
    void create_animation(sf::Vector2f pos,  
sf::Vector2f size, std::string name, float turn_angle  
= 0.f);  
    void update(int time_passed);  
    void draw();
```